

Anexo de Atributos: Diseño (DI) y Aprendizaje Continuo (AC)

Proyecto: spaCEinvaders

Curso: Paradigmas de Programación

Lenguajes utilizados: Java y C

Andres Blanco Coto

Fabiana Moreno Castrillo

Gabriel Bolaños Barboza

Atributo de Diseño (DI)

1.1 Listado de requerimientos del sistema

El proyecto spaCEinvaders consiste en el desarrollo de una aplicación cliente-servidor inspirada en el juego Space Invaders, en la cual se integran dos paradigmas de programación: el paradigma orientado a objetos mediante Java para el servidor y el paradigma imperativo mediante C para el cliente. A partir de la especificación del proyecto y del desarrollo realizado, se identificaron los siguientes requerimientos funcionales y no funcionales.

Requerimientos funcionales:

- El sistema debe implementar un servidor en Java capaz de aceptar conexiones mediante sockets.
- El servidor debe permitir la conexión de al menos dos clientes jugadores.
- El cliente debe estar implementado en lenguaje C.
- El cliente debe conectarse al servidor mediante sockets.
- El servidor debe administrar el estado general del juego, incluyendo vidas, puntos, velocidad, enemigos extraterrestres, OVNI, bunkers y reinicio de partida.
- El cliente debe recibir información del estado del juego enviada por el servidor.
- El cliente debe enviar mensajes al servidor cuando el jugador realice acciones relevantes, como moverse, disparar o eliminar enemigos.
- El servidor debe controlar la lógica principal del juego para evitar inconsistencias entre clientes.
- El sistema debe aplicar programación orientada a objetos en Java mediante clases, paquetes, encapsulamiento y separación de responsabilidades.

- El cliente en C debe utilizar programación imperativa mediante funciones, estructuras, constantes y manejo directo del flujo del programa.
- El sistema debe aplicar al menos dos patrones de diseño válidos en la parte orientada a objetos. Para este proyecto se seleccionaron Observer, Factory y Adapter.
- El sistema debe utilizar listas o estructuras de datos para administrar elementos dinámicos del juego, como clientes conectados, enemigos, disparos o bunkers.

Requerimientos no funcionales:

- El sistema debe ser modular, separando la lógica de red, la lógica del juego y la representación de entidades.
- El sistema debe ser comprensible y mantenible, utilizando nombres claros para clases, métodos, funciones y constantes.
- El sistema debe ser extensible, permitiendo agregar nuevos tipos de enemigos, jugadores o elementos del juego sin modificar excesivamente el código existente.
- El sistema debe manejar errores básicos de conexión, desconexión o cierre de clientes.
- El sistema debe ser eficiente para una cantidad pequeña de clientes, principalmente dos jugadores como mínimo.
- El sistema debe ejecutarse en un entorno de desarrollo local utilizando herramientas disponibles para el grupo, como Java, GCC, MinGW y terminal de Windows.
- El diseño debe considerar el uso razonable de recursos computacionales, evitando crear procesos innecesarios o enviar información excesiva por red.
- El sistema debe promover una distribución clara de responsabilidades entre los integrantes del grupo: servidor Java y lógica central, por un lado, cliente C e interacción del jugador por otro.

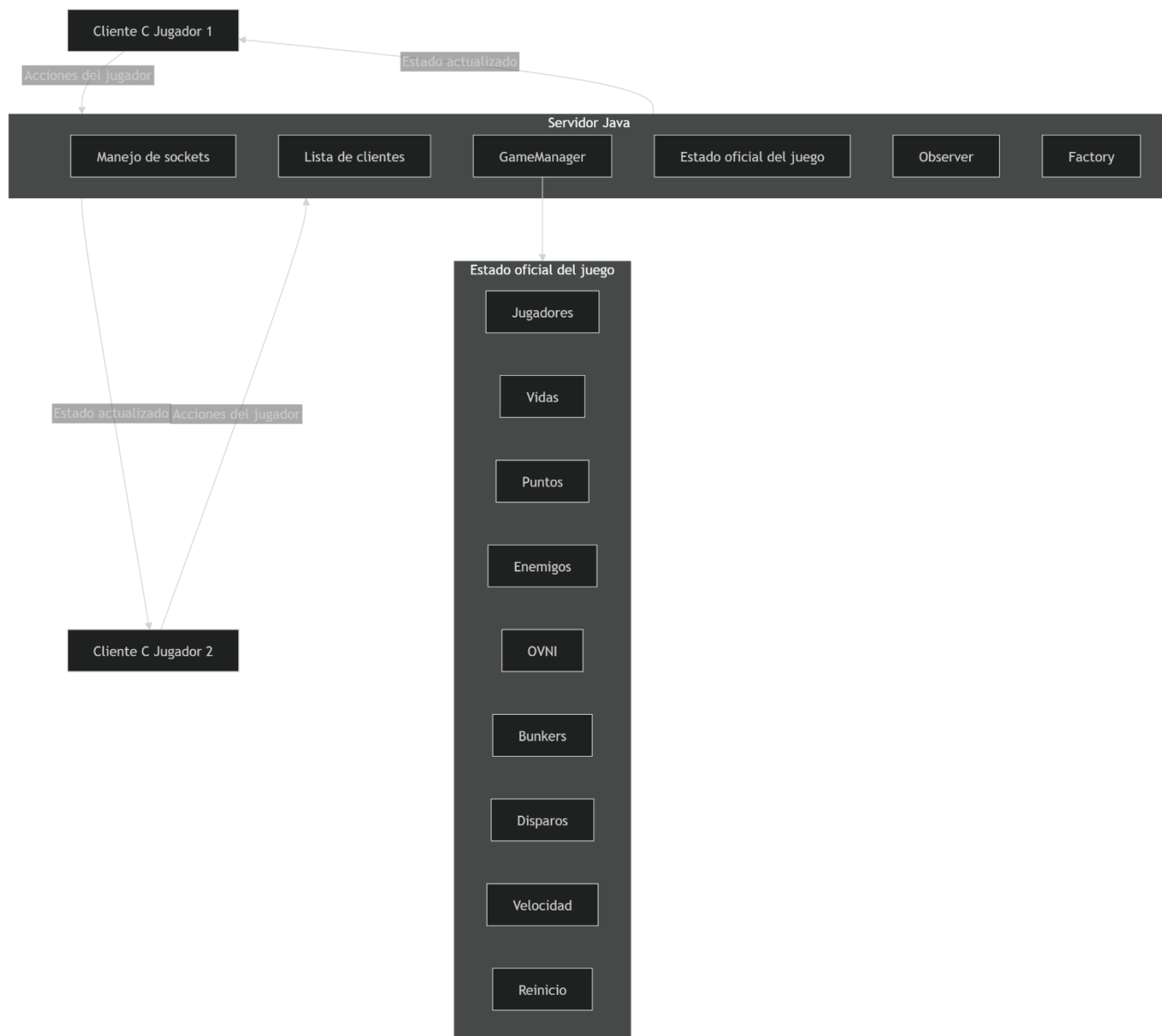
1.2 Alternativas de solución

Para diseñar el sistema se analizaron dos alternativas principales. Ambas permiten cumplir con el objetivo del proyecto, pero presentan diferencias en la organización de responsabilidades, complejidad, escalabilidad y mantenimiento.

Alternativa 1: Servidor centralizado en Java y clientes livianos en C

En esta alternativa, el servidor Java administra la lógica principal del juego. El cliente en C se encarga principalmente de mostrar información, leer acciones del jugador y enviar mensajes al servidor. El servidor conserva el estado oficial de la partida: vidas, puntos, enemigos, disparos, bunkers, velocidad, OVNI y reinicio del juego.

Diagrama de bloques de la alternativa 1:



Ventajas:

Esta alternativa permite mantener el control del juego en un único punto, lo cual reduce problemas de sincronización entre clientes. Además, facilita aplicar orientación a objetos en Java, ya que las entidades del juego pueden representarse mediante clases como Jugador, Enemigo, Ovni, Bunker, Disparo y ServidorJuego. También permite aplicar el patrón Factory para crear enemigos o elementos del juego y Observer para notificar cambios de estado a los clientes conectados.

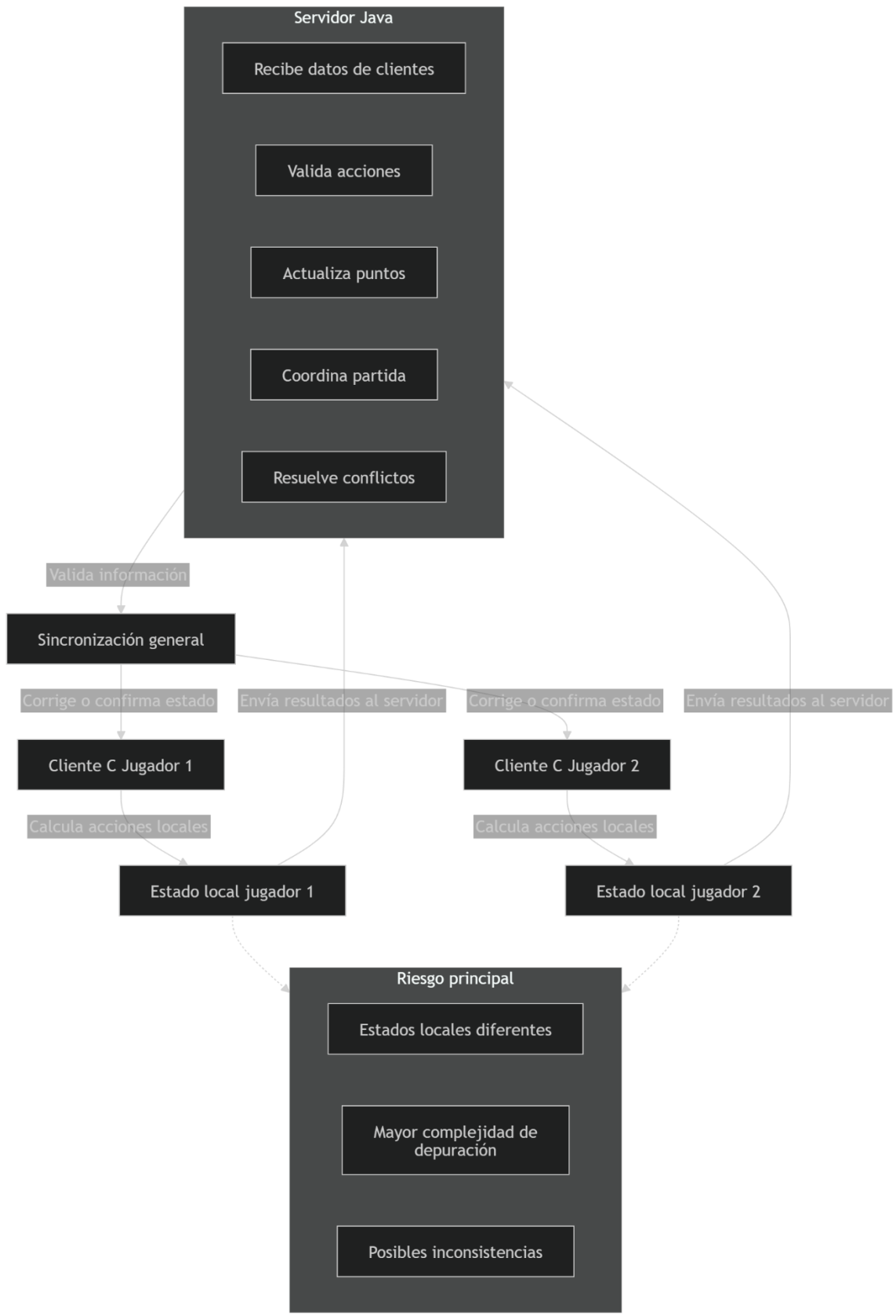
Desventajas:

El servidor tiene mayor responsabilidad y puede volverse más complejo. Además, requiere diseñar correctamente los mensajes entre cliente y servidor para que el cliente pueda interpretar el estado recibido.

Alternativa 2: Lógica distribuida entre servidor y clientes

En esta alternativa, tanto el servidor como los clientes participan en la lógica del juego. El cliente en C podría calcular parte del movimiento, disparos o colisiones, y luego informar al servidor los resultados. El servidor se encargaría de coordinar a los clientes y validar algunos datos.

Diagrama de bloques de la alternativa 2:



Ventajas:

Esta alternativa reduce parte de la carga del servidor y permite que el cliente responda más rápido a ciertas acciones locales. También puede parecer más simple inicialmente para mostrar acciones del jugador en pantalla.

Desventajas:

Puede generar inconsistencias si dos clientes calculan estados distintos. También aumenta la complejidad de validación, porque el servidor tendría que verificar si las acciones reportadas por los clientes son correctas. Además, dificulta mantener una única fuente de verdad del juego y puede complicar el cumplimiento de la parte orientada a objetos del servidor.

1.3 Valoración de opciones de solución

Para seleccionar la mejor alternativa se valoraron criterios técnicos, de recursos, sociales y ambientales aplicables al proyecto.

Criterio 1: Control del estado del juego

Alternativa 1: Alta valoración. El servidor mantiene el estado oficial del juego, por lo que existe una única fuente de verdad.

Alternativa 2: Valoración media. El estado se reparte entre clientes y servidor, lo que puede causar inconsistencias.

Criterio 2: Facilidad para aplicar orientación a objetos

Alternativa 1: Alta valoración. Java puede representar entidades mediante clases y aplicar patrones de diseño de forma natural.

Alternativa 2: Valoración media. Aunque Java sigue presente, parte de la lógica queda en C, lo que reduce la claridad del diseño orientado a objetos.

Criterio 3: Simplicidad de depuración

Alternativa 1: Alta valoración. Al centralizar la lógica en el servidor, los errores se pueden rastrear más fácilmente.

Alternativa 2: Valoración baja. Los errores pueden originarse en el cliente, el servidor o la sincronización entre ambos.

Criterio 4: Uso de recursos

Alternativa 1: Valoración alta. Para un mínimo de dos clientes, el servidor puede manejar la lógica sin un consumo excesivo de recursos.

Alternativa 2: Valoración media. Puede distribuir carga, pero aumenta la cantidad de mensajes y validaciones necesarias.

Criterio 5: Escalabilidad del diseño

Alternativa 1: Valoración alta. Permite agregar nuevos tipos de enemigos o reglas mediante clases y patrones como Factory.

Alternativa 2: Valoración media. Los cambios podrían requerir modificar tanto cliente como servidor.

Criterio 6: Trabajo colaborativo

Alternativa 1: Alta valoración. Permite dividir claramente el trabajo: una persona desarrolla servidor Java y otra desarrolla cliente C.
Alternativa 2: Valoración media. La lógica compartida obliga a coordinar más detalles entre ambas partes.

Criterio 7: Impacto ambiental y recursos computacionales

Alternativa 1: Valoración alta. Al ejecutar localmente y usar comunicación simple por sockets, el consumo computacional es bajo. Además, se reutilizan herramientas gratuitas y disponibles.

Alternativa 2: Valoración media. Puede requerir más pruebas, mayor tiempo de ejecución y más depuración, lo cual implica mayor uso de recursos.

Tabla resumen de valoración:

Criterio	Alternativa 1: Servidor centralizado	Alternativa 2: Lógica distribuida
Control del estado del juego	5	3
Aplicación de POO	5	3
Simplicidad de depuración	5	2
Uso de recursos	4	3
Escalabilidad	5	3
Trabajo colaborativo	5	3
Impacto ambiental y eficiencia	4	3
Total	33	20

Escala utilizada:
1 = muy bajo, 2 = bajo, 3 = medio, 4 = alto, 5 = muy alto.

1.4 Selección de la propuesta final

Se seleccionó la Alternativa 1: servidor centralizado en Java y clientes livianos en C.

La decisión se justifica porque esta alternativa permite cumplir mejor con los requerimientos del proyecto. El servidor Java puede aplicar de forma clara el paradigma orientado a objetos, administrar el estado oficial del juego y utilizar patrones de diseño como Observer y Factory.

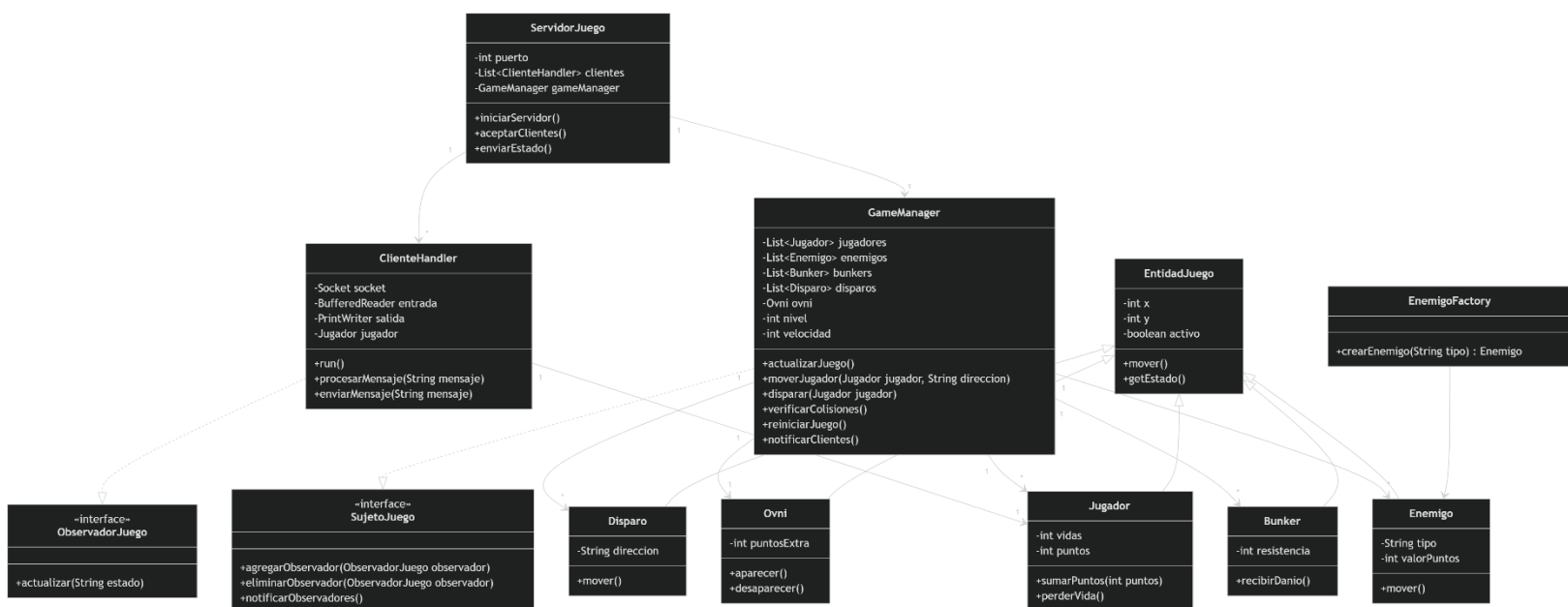
Además, permite que el cliente en C se enfoque en la programación imperativa, utilizando structs, funciones, constantes y comunicación por sockets.

Esta propuesta también facilita la división del trabajo entre los integrantes del grupo, ya que una parte puede concentrarse en el servidor y otra en el cliente. A nivel técnico, reduce el riesgo de inconsistencias entre jugadores y facilita el proceso de prueba y depuración.

1.5 Diseño de la alternativa seleccionada

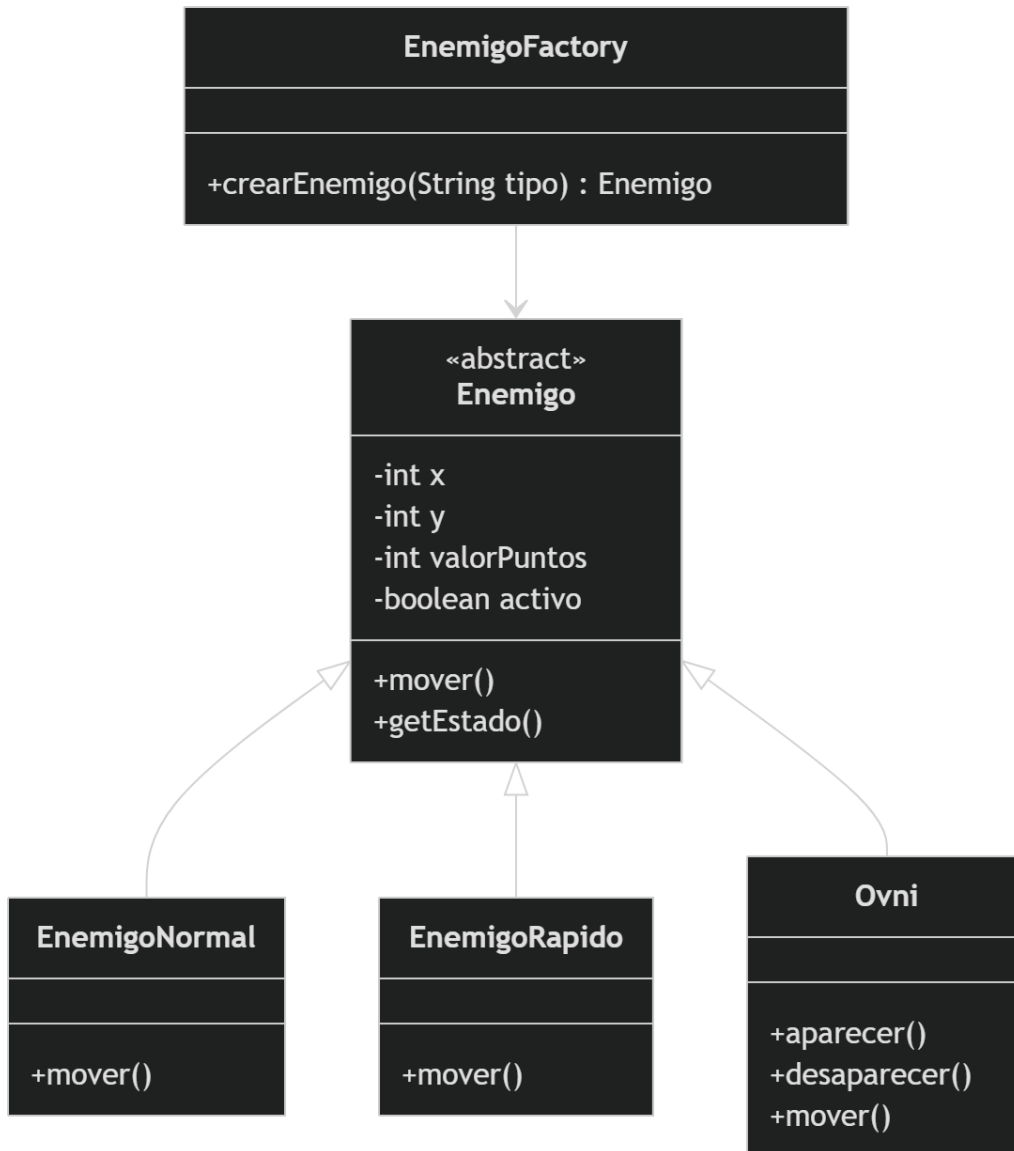
El diseño final se basa en una arquitectura cliente-servidor. El servidor Java funciona como núcleo del sistema y mantiene el estado principal del juego. Los clientes en C se conectan mediante sockets, envían acciones del jugador y reciben actualizaciones del estado.

Diagrama general de bloques del servidor Java:



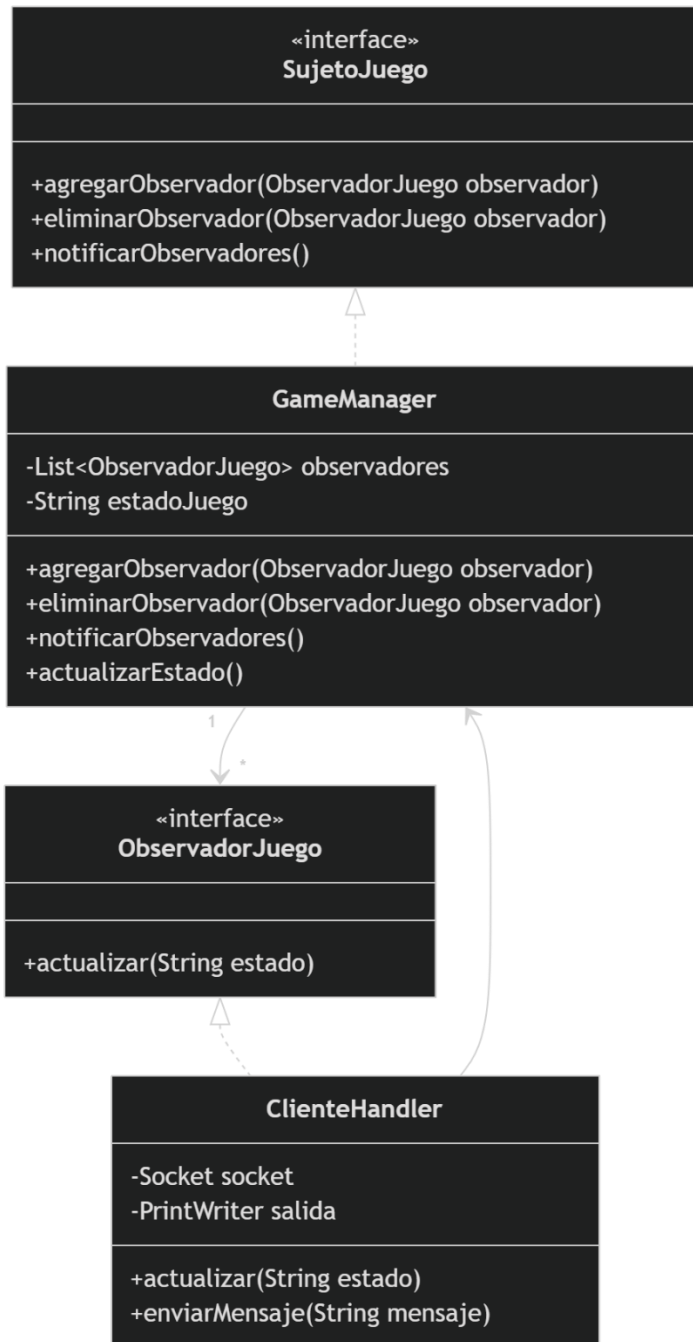
Aplicación del patrón Factory:

El patrón Factory se utiliza para crear objetos del juego sin instanciarlos directamente desde cualquier parte del código. Por ejemplo, se puede tener una clase EnemigoFactory encargada de crear enemigos normales, enemigos rápidos u OVNI's. Esto mejora la extensibilidad del sistema, ya que en el futuro se pueden agregar nuevos tipos de enemigos sin modificar la lógica principal del servidor.



Aplicación del patrón Observer:

El patrón Observer se utiliza para notificar a los clientes cuando el estado del juego cambia. El servidor actúa como sujeto observado y los clientes conectados funcionan como observadores. Cuando ocurre un cambio importante, como movimiento, disparo, eliminación de enemigo, pérdida de vida o reinicio de partida, el servidor envía el nuevo estado a los clientes.

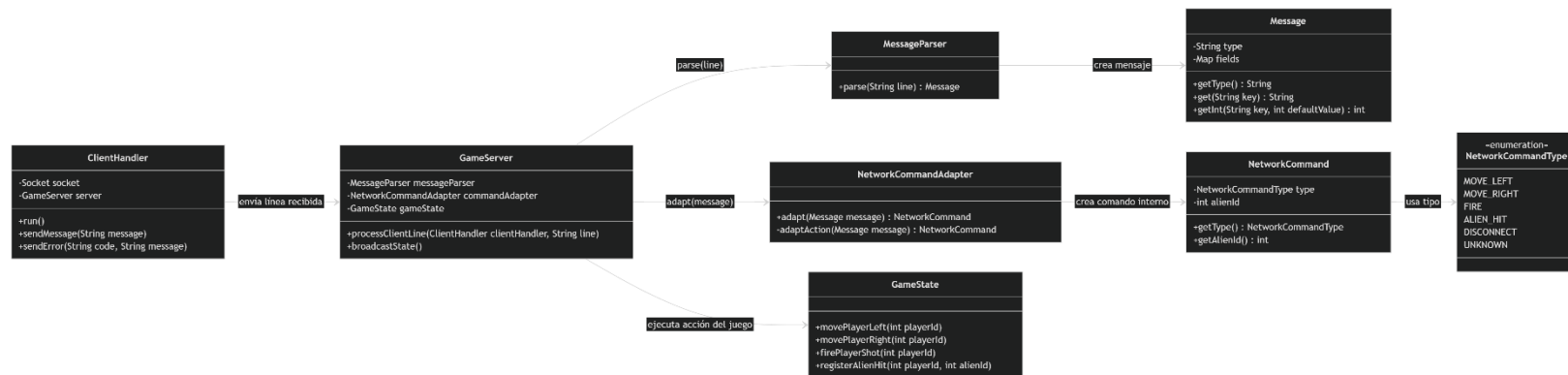


Aplicación del patrón Adapter

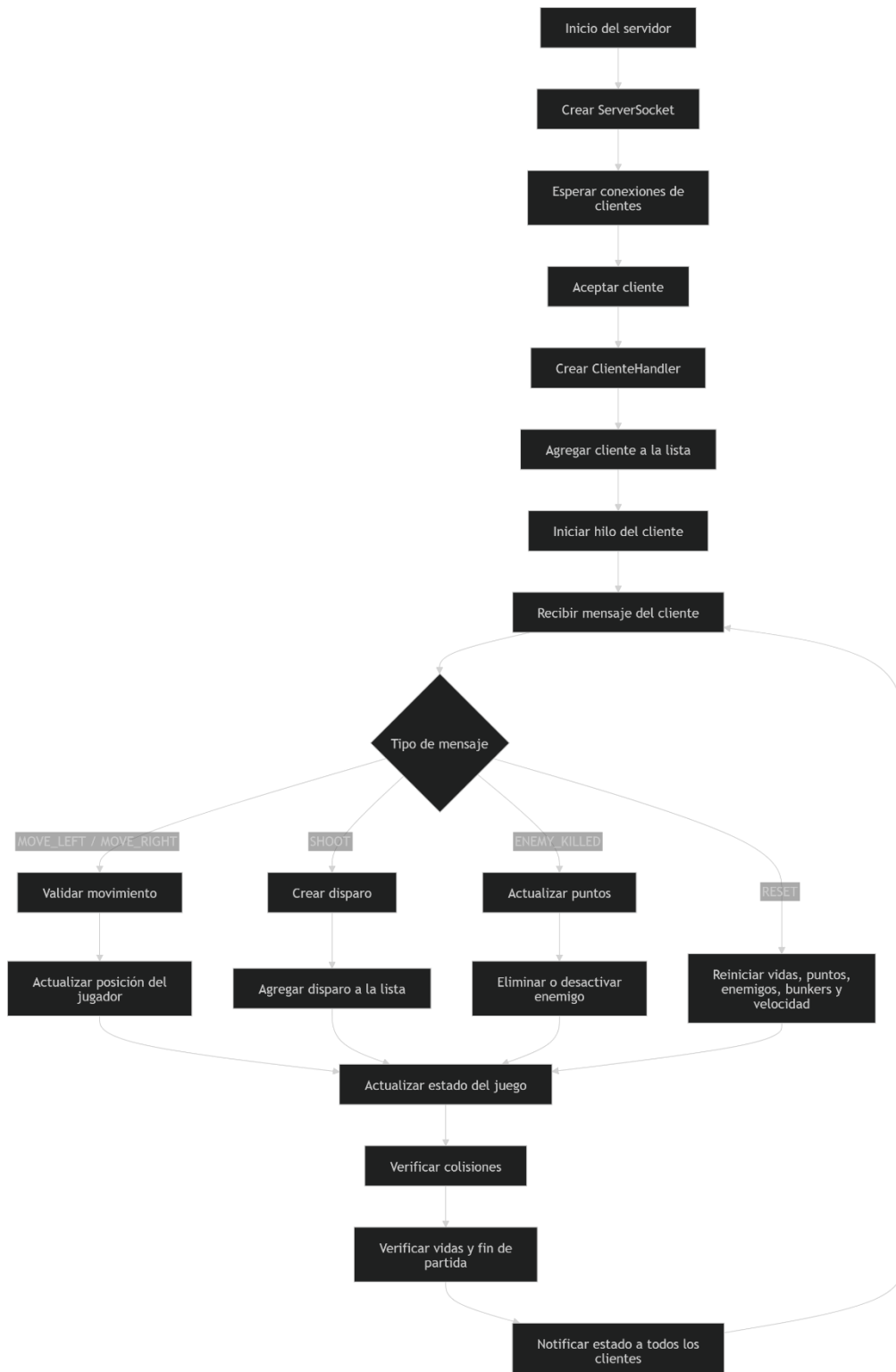
Además de los patrones Observer y Factory, el servidor Java utiliza el patrón Adapter para separar el protocolo de comunicación de la lógica interna del juego. En el proyecto, los clientes envían mensajes de texto por sockets, por ejemplo `ACTION|cmd=MOVE_LEFT`, `ACTION|cmd=FIRE` o `ALIEN_HIT|alienId=5`. Estos mensajes no son ejecutados directamente por el servidor, sino que primero son interpretados por `MessageParser` y luego

convertidos por NetworkCommandAdapter en comandos internos de tipo NetworkCommand.

De esta forma, el adaptador transforma mensajes externos de red en acciones internas comprensibles para el servidor, como MOVE_LEFT, MOVE_RIGHT, FIRE, ALIEN_HIT o DISCONNECT. Posteriormente, GameServer utiliza esos comandos para invocar los métodos correspondientes en GameState, como mover al jugador, disparar o registrar un impacto.



Flujo principal del servidor:



Algoritmos principales:

Algoritmo de conexión de clientes:

1. El servidor abre un puerto de escucha.
2. Cada cliente intenta conectarse al servidor mediante la dirección IP y el puerto.
3. El servidor acepta la conexión.
4. El servidor crea un manejador para ese cliente.
5. El cliente queda registrado en una lista de clientes activos.
6. El servidor puede enviar actualizaciones a todos los clientes conectados.

Algoritmo de movimiento:

1. El cliente detecta una tecla de movimiento.
2. El cliente envía un mensaje al servidor, por ejemplo MOVE_LEFT o MOVE_RIGHT.
3. El servidor recibe el mensaje.
4. El servidor valida que el movimiento esté dentro de los límites permitidos.
5. El servidor actualiza la posición del jugador.
6. El servidor notifica el nuevo estado a los clientes.

Algoritmo de disparo:

1. El cliente detecta la acción de disparar.
2. El cliente envía el mensaje SHOOT al servidor.
3. El servidor crea un disparo asociado al jugador.
4. El servidor actualiza la posición del disparo en cada ciclo del juego.
5. El servidor verifica colisiones entre disparos y enemigos.
6. Si hay colisión, se elimina o desactiva el enemigo.
7. Se actualizan los puntos del jugador.
8. El servidor notifica el cambio de estado.

Algoritmo de reinicio:

1. El servidor detecta condición de fin de partida o recibe solicitud de reinicio.
2. Se restablecen vidas, puntos, enemigos, bunkers, OVNI y velocidad.
3. Se limpian las listas de disparos activos.
4. Se reposicionan los jugadores.
5. Se notifica a todos los clientes que la partida fue reiniciada.

1.6 Validación del diseño

Para validar el diseño final se utilizó una lista de verificación basada en los requerimientos definidos.

Requerimiento	Cumplimiento	Evidencia
Servidor implementado en Java	Cumple	Se desarrolló la estructura del servidor con sockets en Java.

Requerimiento	Cumplimiento	Evidencia
Cliente implementado en C	Cumple	Se creó el cliente en C y se compiló con GCC.
Comunicación mediante sockets	Cumple	El cliente se conecta al servidor y recibe/imprime información correctamente.
Aceptar mínimo dos clientes	En cumple según prueba final	El diseño permite manejar múltiples clientes mediante lista e hilos.
Manejo de vidas	Cumple en diseño	GameManager contempla atributo de vidas por jugador.
Manejo de puntos	Cumple en diseño	Jugador registra puntos y el servidor los actualiza.
Manejo de velocidad	Cumple en diseño	GameManager administra velocidad de enemigos o juego.
Manejo de extraterrestres	Cumple en diseño	Lista de enemigos administrada por el servidor.
Manejo de OVNI	Cumple en diseño	Entidad Ovni creada como elemento especial.
Manejo de bunkers	Cumple en diseño	Lista de bunkers con resistencia.
Reinicio del juego	Cumple en diseño	Algoritmo de reinicio restablece estado.
Uso de POO en Java	Cumple	Se diseñan clases como Jugador, Enemigo, Bunker, Ovni y GameManager.
Uso de programación imperativa en C	Cumple	El cliente usa funciones, structs, constantes y flujo secuencial.
Uso de patrones de diseño	Cumple	Se aplican Observer y Factory.
Uso de listas o estructuras de datos	Cumple	Se usan listas para clientes, enemigos, disparos y bunkers.
Separación de responsabilidades	Cumple	Cliente maneja entrada/salida; servidor maneja lógica central.

Conclusión de validación:

El diseño seleccionado cumple con los requerimientos principales del proyecto y permite integrar adecuadamente los paradigmas imperativo y orientado a objetos. La arquitectura cliente-servidor centralizada reduce la complejidad de sincronización, facilita las pruebas y permite una mejor organización del trabajo. Además, el uso de patrones de diseño aporta extensibilidad y claridad al servidor Java.

Atributo de Aprendizaje Continuo (AC)

2.1 Identificación de necesidades de aprendizaje

Durante el desarrollo del proyecto se identificaron varias necesidades de aprendizaje relacionadas con conocimientos técnicos, habilidades prácticas y actitudes necesarias para completar el sistema.

Necesidades de conocimiento:

1. Comprender la comunicación por sockets entre un servidor Java y un cliente C.
2. Reforzar el uso de programación orientada a objetos en Java.
3. Comprender la creación y administración de hilos para atender múltiples clientes.
4. Aplicar correctamente patrones de diseño, especialmente Observer, Factory y Adapter.
5. Reforzar la programación imperativa en C mediante funciones, structs y archivos de constantes.
6. Comprender cómo enviar y recibir mensajes entre programas escritos en lenguajes diferentes.
7. Aprender a compilar programas C en Windows usando GCC y la librería `ws2_32`.
8. Comprender cómo dividir responsabilidades entre cliente y servidor.

Necesidades de habilidades:

1. Depurar errores de compilación en C.
2. Interpretar errores relacionados con librerías, includes y sockets.
3. Ejecutar programas desde terminal.
4. Probar conexiones cliente-servidor de forma local.
5. Organizar archivos del proyecto en carpetas claras.
6. Diseñar diagramas que representen la arquitectura del sistema.
7. Documentar decisiones de diseño con criterios técnicos.

Necesidades de actitudes:

1. Mantener disposición para aprender tecnologías que no se dominaban completamente.
2. Ser paciente ante errores de compilación o conexión.
3. Buscar soluciones de forma autónoma antes de descartar una alternativa.
4. Coordinar el trabajo con la compañera encargada del cliente en C.
5. Evaluar críticamente si el diseño realmente cumple con los requerimientos de la tarea.

2.2 Tecnologías nuevas y emergentes que contribuyeron al aprendizaje

Para el desarrollo del proyecto se utilizaron o reforzaron tecnologías y herramientas relevantes para el aprendizaje continuo en ingeniería de software.

1. Sockets TCP

Los sockets TCP permitieron comprender cómo dos programas pueden comunicarse a través de red, incluso cuando están escritos en lenguajes diferentes. Esta tecnología es fundamental en sistemas distribuidos, videojuegos multijugador, aplicaciones web y servicios cliente-servidor.

2. Java ServerSocket

El uso de ServerSocket en Java permitió implementar el servidor del juego. Esta tecnología ayudó a comprender cómo se reciben conexiones, cómo se aceptan clientes y cómo se puede administrar comunicación concurrente.

3. Winsock en C

En Windows, el cliente C requiere el uso de la librería ws2_32 para trabajar con sockets. Esto permitió aprender detalles específicos del sistema operativo y comprender que la programación de red puede variar según el entorno.

4. GCC y MinGW

El compilador GCC mediante MinGW permitió compilar el cliente C en Windows. Esta herramienta contribuyó al aprendizaje práctico sobre compilación, enlaces de librerías y generación de ejecutables.

5. Programación concurrente con hilos

La atención de varios clientes requiere comprender el uso de hilos o procesos de ejecución independientes. Esta tecnología es importante en aplicaciones modernas donde varios usuarios o tareas deben atenderse al mismo tiempo.

6. Patrones de diseño

Los patrones Observer y Factory permitieron relacionar conceptos teóricos de diseño de software con una implementación práctica. Estos patrones son ampliamente utilizados en sistemas modernos porque mejoran la organización, extensibilidad y mantenimiento del código.

7. Inteligencia artificial como apoyo de aprendizaje

Se utilizó la IA como herramienta de apoyo para aclarar errores, revisar estructura del código, comprender mensajes de compilación y generar explicaciones paso a paso. Su uso se aplicó como apoyo formativo, no como sustituto del trabajo del grupo.

2.3 Acciones o estrategias implementadas para solventar las necesidades de aprendizaje

Para atender las necesidades identificadas se implementaron varias estrategias de aprendizaje y trabajo.

Estrategia 1: Pruebas incrementales

En lugar de desarrollar todo el sistema completo de una sola vez, se realizaron pruebas pequeñas e incrementales. Primero se verificó que el cliente C compilara correctamente. Luego se revisó que generara el archivo ejecutable. Después se probó que el cliente pudiera correr e imprimir información en la terminal. Esta estrategia permitió detectar errores de manera más controlada.

Evidencia:

El cliente C logró compilar correctamente utilizando GCC y se generó un archivo client.exe. Además, el programa logró ejecutarse e imprimir información en la terminal, aunque inicialmente se cerraba después de unos segundos. Esto permitió identificar que la compilación estaba funcionando y que el siguiente paso era revisar la permanencia de la conexión y la lógica del ciclo principal.

Estrategia 2: Separación de responsabilidades

Se dividió el trabajo del proyecto en dos partes principales. Una persona se encargó del servidor Java, los sockets, el estado del juego, los patrones de diseño y la lógica central. La otra persona se encargó del cliente C, la visualización, el movimiento, disparos, structs, constantes y generación del ejecutable.

Evidencia:

La división permitió trabajar en paralelo y mantener más claro qué parte correspondía al paradigma orientado a objetos y qué parte correspondía al paradigma imperativo.

Estrategia 3: Repaso de conceptos de sockets

Se repasó el funcionamiento básico de sockets, incluyendo servidor, cliente, puerto, dirección IP, envío y recepción de mensajes. Esto fue necesario porque el proyecto requería comunicación entre Java y C.

Evidencia:

Se logró comprender que el servidor debe ejecutarse primero, abrir un puerto y esperar conexiones. Luego el cliente C debe conectarse a ese puerto para enviar y recibir mensajes.

Estrategia 4: Uso de patrones de diseño

Se analizaron patrones que realmente aportaran al proyecto. Se seleccionaron Observer y Factory porque se adaptan a la lógica del juego. Observer permite notificar cambios del estado a los clientes y Factory permite crear enemigos u objetos del juego de forma ordenada.

Evidencia:

El diseño final incluye Observer para la comunicación de cambios y Factory para la creación de entidades del juego.

Estrategia 5: Organización del proyecto por carpetas

Se propuso mantener una organización clara de archivos, separando código fuente, includes, constantes y ejecutables. Esto facilita compilar, revisar y entregar el proyecto.

Evidencia:

En el cliente C se trabajó con carpetas como src e include, además de compilar desde terminal con los archivos correspondientes.

Estrategia 6: Uso de apoyo tecnológico para depuración y explicación

Se utilizó asistencia tecnológica para interpretar errores de compilación, revisar comandos y entender cómo continuar el desarrollo. Esta estrategia permitió acelerar el aprendizaje y reducir bloqueos durante el trabajo.

Evidencia:

Se revisó el comando de compilación del cliente C y se identificó la importancia de enlazar la librería ws2_32 en Windows para sockets.

2.4 Evaluación crítica y cuantitativa de la eficacia de las estrategias

Para evaluar la eficacia de las estrategias implementadas, se utilizó una escala de 1 a 5:

1 = No fue eficaz
 2 = Poco eficaz
 3 = Medianamente eficaz
 4 = Eficaz
 5 = Muy eficaz

Estrategia	Necesidad atendida	Resultado observado	Calificación
Pruebas incrementales	Depuración y validación del cliente C	Permitió confirmar que el cliente compilaba y generaba ejecutable.	5
Separación de responsabilidades	Organización del trabajo grupal	Facilitó dividir servidor Java y cliente C.	5
Repaso de sockets	Comprensión de comunicación cliente-servidor	Ayudó a entender el orden de ejecución servidor-cliente.	4
Uso de patrones de diseño	Aplicación de POO avanzada	Permitió justificar Observer y Factory en el diseño.	4
Organización por carpetas	Mantenibilidad del proyecto	Mejóro la claridad del proyecto y la compilación.	4
Uso de apoyo tecnológico	Solución de errores y aprendizaje autónomo	Ayudó a interpretar errores y avanzar más rápido.	4

Promedio cuantitativo:

$$\begin{aligned} \text{Promedio} &= (5 + 5 + 4 + 4 + 4 + 4) / 6 \\ \text{Promedio} &= 26 / 6 \\ \text{Promedio} &= 4.33 / 5 \end{aligned}$$

Interpretación:

El resultado obtenido fue 4.33 de 5, lo cual indica que las estrategias implementadas fueron eficaces para atender las necesidades de aprendizaje del proyecto. Las estrategias más efectivas fueron las pruebas incrementales y la separación de responsabilidades, ya que permitieron avanzar de forma ordenada y confirmar progresos concretos.

Análisis crítico:

Aunque las estrategias fueron efectivas, se identificó que el aprendizaje de sockets entre Java y C requiere más práctica, especialmente para mantener la conexión activa y definir un protocolo de mensajes claro. También se observó que el uso de patrones de diseño debe justificarse desde el diseño y no agregarse únicamente al final, ya que su valor está en mejorar la estructura del sistema.

En un contexto de cambio tecnológico, este proyecto permitió reforzar la capacidad de adaptación al trabajar con dos lenguajes distintos, herramientas de compilación, comunicación por red y arquitectura cliente-servidor. Además, permitió comprender que aprender de forma continua no consiste únicamente en buscar información, sino en aplicar, probar, corregir y evaluar críticamente los resultados obtenidos.

Conclusión del atributo AC:

El desarrollo del proyecto spaCEinvaders permitió identificar necesidades reales de aprendizaje relacionadas con programación en red, programación orientada a objetos, programación imperativa, patrones de diseño y trabajo colaborativo. Las estrategias aplicadas permitieron avanzar de forma efectiva y demostraron una actitud de aprendizaje continuo. La evaluación cuantitativa mostró un promedio de 4.33/5, lo cual evidencia que las acciones implementadas fueron adecuadas para enfrentar los retos técnicos del proyecto.

Bibliografía:

Apache Maven Project. (s. f.). Maven getting started guide. Apache Maven. Recuperado el 15 de junio de 2026, de <https://maven.apache.org/guides/getting-started/>

Chacon, S., & Straub, B. (2014). Pro Git (2.^a ed.). Apress. <https://git-scm.com/book/en/v2>

Espressif Systems. (s. f.). Universal asynchronous receiver/transmitter (UART). ESP-IDF Programming Guide. Recuperado el 15 de junio de 2026, de <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html>

Free Software Foundation. (s. f.). GCC online documentation. GNU Project. Recuperado el 15 de junio de 2026, de <https://gcc.gnu.org/onlinedocs/>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Microsoft. (2021, 12 de octubre). socket function (winsock2.h). Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-socket>

Microsoft. (2023, 9 de febrero). Getting started with Winsock. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/winsock/getting-started-with-winsock>

MSYS2. (s. f.). Package management. MSYS2 Documentation. Recuperado el 15 de junio de 2026, de <https://www.msys2.org/docs/package-management/>

Oracle. (s. f.). Class ServerSocket. Java Platform, Standard Edition 17 API Specification. Recuperado el 15 de junio de 2026, de <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/ServerSocket.html>

Oracle. (s. f.). Class Socket. Java Platform, Standard Edition 17 API Specification. Recuperado el 15 de junio de 2026, de <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/Socket.html>

raylib. (s. f.). raylib cheatsheet. Recuperado el 15 de junio de 2026, de <https://www.raylib.com/cheatsheet/cheatsheet.html>

Taito Corporation. (s. f.). The evolution of Space Invaders. Google Arts & Culture. Recuperado el 15 de junio de 2026, de https://artsandculture.google.com/story/the-evolution-of-space-invaders/wAVxCu_0BaX0aw